

Московский авиационный институт
(национальный исследовательский университет)

Факультет Компьютерные науки и прикладная математика

Кафедра вычислительной математики и программирования

Лабораторная работа №5 по курсу «Дискретный анализ»

Студент: А.С Савельев
Преподаватель: А. А. Кухтичев
Группа: М8О-206БВ-24
Дата: 25 мая 2026 г.
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №5

Условие задачи:

Требуется разработать программу для определения лексикографически минимальной циклической ротации заданной строки, используя суффиксное дерево.

Вариант алгоритма: Суффиксное дерево с оптимальным поиском минимальной ротации через обход дерева.

Входные данные: Одна строка, содержащая исходную последовательность символов.

Выходные данные: Лексикографически минимальная циклическая ротация исходной строки длиной, равной длине входной строки.

1 Описание

Требуется реализовать алгоритм поиска лексикографически минимальной циклической ротации строки с использованием суффиксного дерева.

1.1 Основная идея

Суффиксное дерево - это сжатое дерево всех суффиксов строки. Для решения задачи о минимальной циклической ротации применяется классический приём: строится суффиксное дерево для строки $s + s + \$$ (где $\$$ — non vocab символ), что позволяет рассмотреть все циклические сдвиги исходной строки как суффиксы этой конкатенации.

Для строки длины n существует n циклических ротаций. Наивный подход требует $O(n^2 \log n)$ времени для сравнения всех вариантов. Использование суффиксного дерева позволяет достичь линейной сложности $O(n)$ при поиске оптимальной ротации.

1.2 Структура суффиксного дерева

Каждый узел дерева содержит:

- l, r — границы подстроки в исходной строке
- `next` — словарь переходов по символам
- `slink` — суффиксная ссылка для оптимизации построения

Суффиксная ссылка соединяет узел, соответствующий подстроке $s\alpha$, с узлом, соответствующим подстроке α (где s — символ, α — подстрока).

1.3 Построение суффиксного дерева (алгоритм Укконена)

Алгоритм Укконена строит суффиксное дерево за $O(n)$ времени в два прохода:

1. **Расширение:** На каждом шаге добавляется новый суффикс исходной строки
2. **Оптимизация через суффиксные ссылки:** При несовпадении используются суффиксные ссылки вместо возврата в корень

Ключевые параметры алгоритма:

- `act_v` — активный узел
- `act_pos` — позиция на ребре активного узла
- `act_len` — длина активного пути
- `rem` — количество оставшихся расширений

1.4 Поиск минимальной ротации в дереве

После построения суффиксного дерева для $s + s + \$$ выполняется поиск оптимального пути от корня:

1. Начинаем с корня дерева
2. На каждом шаге выбираем лексикографически первый переход (исключая $\$$)
3. Берём из каждого ребра количество символов, необходимых для получения строки длины $|s|$
4. Останавливаемся, когда накоплено ровно $|s|$ символов

Это гарантирует лексикографическую минимальность, так как мы всегда выбираем наименьший доступный символ на каждом уровне.

2 Исходный код

2.1 Структура узла

```
1 struct Node {
2     int l, r;
3     int slink;
4     map<char, int> next;
5
6     Node(int l, int r) : l(l), r(r), slink(-1) {}
7 };
```

2.2 Вспомогательная функция

```
1 int edge_length(int v) const {
2     if (v == 0) return 0;
3
4     int right = (nodes[v].r == -1) ? leaf_end : nodes[v].r;
5
6     return right - nodes[v].l + 1;
7 }
```

Функция вычисляет длину ребра, ведущего в узел v . Для листовых узлов используется переменная `leaf_end`, которая динамически обновляется во время построения.

2.3 Построение дерева

Основной цикл построения суффиксного дерева:

```
1 for (int i = 0; i < text.length(); ++i) {
2     extend(i);
3 }
```

На каждом шаге i к дереву добавляются все суффиксы, начинающиеся с позиции 0 и далее, которые заканчиваются в позиции i .

2.4 Поиск оптимальной ротации

```
1 string optimal_lecs_split(int s_len) const {
2     int curr = 0;
3     int len = 0;
4     string res;
5     res.reserve(s_len);
6
7     while (len < s_len) {
8         int next_el = -1;
9
10        for (auto const& [c, v_idx] : nodes[curr].next) {
11            if (c == '$') continue;
12            next_el = v_idx;
13            break;
14        }
15    }
```

```

16         if (next_el == -1) break;
17
18         int elen = edge_length(next_el);
19         int take = min(elen, s_len - len);
20
21         res += text.substr(nodes[next_el].l, take);
22         len += take;
23         curr = next_el;
24     }
25     return res;
26 }

```

Функция выбирает лексикографически минимальный переход на каждом узле и накапливает символы до достижения длины исходной строки.

2.5 Главная функция решения

```

1 string solve(const string& s) {
2     if (s.empty()) return "";
3
4     string t = s + s + "$";
5     SuffixTree st(t);
6
7     return st.optimal_lecs_split(s.size());
8 }

```

Конкатенация $s + s$ обеспечивает наличие всех циклических ротаций как суффиксов новой строки. Завершающий символ $\$$ предотвращает нежелательные совпадения.

Сложность: Построение дерева — $O(n)$, поиск минимальной ротации — $O(n)$. Итоговая сложность: $O(n)$.

3 Тест производительности

Тест производительности сравнивает время работы реализованного алгоритма на основе суффиксного дерева с наивным подходом перебора всех циклических ротаций с использованием STL функций. Наивный метод генерирует все n возможных ротаций и находит минимальную через `std::min_element`.

Тестовые данные генерируются Python-скриптом с различными параметрами: короткие строки (до 1000 символов), средние (до 10000 символов) и длинные (до 100000 символов). Время измеряется в микросекундах (us) с использованием библиотеки `<chrono>`.

3.1 Результаты базовых бенчмарков

```
1 artems@MacBook-Air-artem-3 lab5 % ./benchmark
2
3 String length: 1000
4 -----
5 Suffix Tree: 831 us
6 Naive std::min_element: 952 us
7 -----
8 Speedup: 1.14561x
9
10 artems@MacBook-Air-artem-3 lab5 % python3 test_generator.py 10000
    random | ./benchmark
11
12 String length: 10000
13 -----
14 Suffix Tree: 3486 us
15 Naive std::min_element: 25357 us
16 -----
17 Speedup: 7.27395x
18
19 String length: 50000
20 -----
21 Suffix Tree: 25256 us
22 Naive std::min_element: 379677 us
23 -----
24 Speedup: 15.0331x
```

3.2 Анализ результатов

3.2.1 Растущее преимущество с увеличением размера

Результаты демонстрируют, что разница в производительности растет экспоненциально с увеличением длины строки:

1. **Строки длины 100:** Suffix tree примерно такой же по производительности, что и наивный подход (1.14x быстрее)
2. **Строки длины 1000:** Преимущество возрастает до 7x
3. **Строки длины 50000:** Превосходство достигает 15x

Это объясняется асимптотической сложностью:

- **Suffix Tree:** $O(n)$ построение + $O(n)$ поиск + $O(1)$ вхождение = $O(n)$
- **Naive approach:** $O(n)$ ротаций $\times O(n)$ сравнение = $O(n^2)$

3.2.2 Влияние повторяющихся паттернов

Специальный тест на строке, состоящей из повторяющегося паттерна:

```

1 Test 4: Repetitive pattern (aaabaaabaaab...)
2 artems@MacBook-Air-artem-3 lab5 % python3 test_generator.py 50000
   worst_case | ./benchmark
3
4 String length: 50000
5 -----
6 Suffix Tree: 12060 us
7 Naive std::min_element: 477186 us
8 -----
9 Speedup: 39.5677x

```

Даже на повторяющихся данных suffix tree сохраняет линейную сложность, в то время как наивный подход остаётся $O(n^2)$.

3.2.3 Критическая точка пересечения

Анализ показывает, что реализация suffix tree ощущается намного выгоднее наивного подхода начиная с длины строки примерно 200 символов. Для строк меньшего размера наивный подход может быть достаточным, учитывая простоту реализации.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ*, 3-е издание. — Издательский дом «Вильямс», 2013. — 1328 с.
- [2] Укконен, Э. *On-line construction of suffix trees*. *Algorithmica*, Vol. 14, No. 3, 1995. — С. 249–260.
- [3] ИТМО Академия *Суффиксные деревья и их применение*. URL: https://neerc.ifmo.ru/wiki/index.php?title=Суффиксное_дерево
- [4] *C++ reference*.
URL: https://en.cppreference.com/w/cpp/algorithm/min_element

4 Выводы

Выполнив пятую лабораторную работу, я глубоко изучил теорию и практику суффиксных деревьев, а также применил полученные знания для решения задачи о лексикографически минимальной циклической ротации строки.

В процессе выполнения лабораторной работы я:

- Полностью разобрал алгоритм Укконена построения суффиксного дерева за линейное время, включая механизм активного состояния и суффиксные ссылки.
- Реализовал эффективный поиск оптимальной ротации через обход дерева с выбором лексикографически минимального перехода на каждом шаге.
- Применил классический приём конкатенации $s + s + \$$ для представления всех циклических ротаций как суффиксов в одной структуре.
- Провел бенчмарки, демонстрирующие практическое превосходство suffix tree ($O(n)$) над наивным подходом ($O(n^2)$) на строках разной длины.

Реализация демонстрирует корректное понимание структур данных на строках, эффективный алгоритм построения суффиксного дерева и применение этой структуры для решения практических задач оптимизации. Полученные навыки применимы к широкому спектру задач обработки текстов, включая поиск наибольшей общей подстроки, определение повторяемых элементов и анализ строковых данных.